Tasks

Task 1: Solve the following programming problems:

1. STL Container Practice: Write a program using STL containers that:
    1. Uses `vector<string>` to store names
    2. Uses `map<string, int>` to store age against each name
    3. Implements functions to:
        1. Add new name-age pair
        2. Find all people above certain age
        3. Sort and display names alphabetically

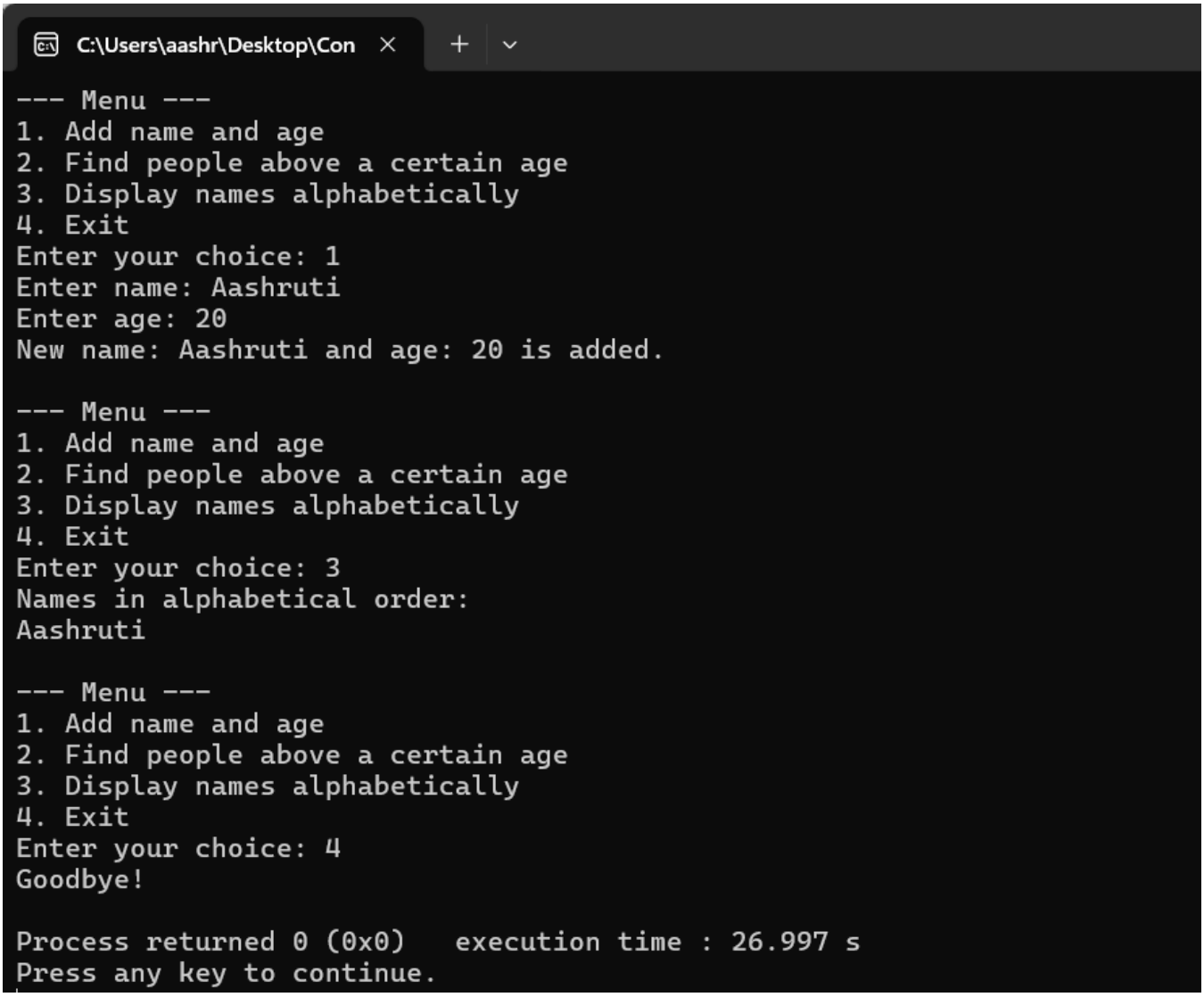
```
#include <iostream>
#include <vector>
#include <map>
#include <algorithm>

using namespace std;

void addPerson(vector<string>& names, map<string, int>& ageMap) {
    string name;
    int age;

    cout << "Enter name: ";
    cin.ignore();
    getline(cin, name);

    cout << "Enter age: ";
    cin >> age;

    names.push_back(name);
```

```
    ageMap[name] = age;

    cout << "New name: " << name << " and age: " << age << " is added.\n";
}

void findPeopleAboveAge(const map<string, int>& ageMap) {
    int threshold;
    cout << "Enter age threshold: ";
    cin >> threshold;

    bool found = false;
    cout << "People above age " << threshold << ":\n";
    for (const auto& pair : ageMap) {
        if (pair.second > threshold) {
            cout << pair.first << " (" << pair.second << " years old)\n";
            found = true;
        }
    }

    if (!found) {
        cout << "No one found above age " << threshold << ".\n";
    }
}

void displaySortedNames(const vector<string>& names) {
    vector<string> sortedNames = names;
    sort(sortedNames.begin(), sortedNames.end());
```

```
    cout << "Names in alphabetical order:\n";
    for (const string& name : sortedNames) {
        cout << name << endl;
    }
}

int main() {
    vector<string> names;
    map<string, int> ageMap;

    int choice;

    do {
        cout << "\n--- Menu ---\n";
        cout << "1. Add name and age\n";
        cout << "2. Find people above a certain age\n";
        cout << "3. Display names alphabetically\n";
        cout << "4. Exit\n";
        cout << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                addPerson(names, ageMap);
                break;
            case 2:
```

```
            findPeopleAboveAge(ageMap);
            break;
        case 3:
            displaySortedNames(names);
            break;
        case 4:
            cout << "Goodbye!\n";
            break;
        default:
            cout << "Invalid choice. Please try again.\n";
    }

  } while (choice != 4);

  return 0;
}
```

Output:


C:\Users\aashr\Desktop\Con
--- Menu ---
1. Add name and age
2. Find people above a certain age
3. Display names alphabetically
4. Exit
Enter your choice: 1
Enter name: Aashruti
Enter age: 20
New name: Aashruti and age: 20 is added.
--- Menu ---
1. Add name and age
2. Find people above a certain age
3. Display names alphabetically
4. Exit
Enter your choice: 3
Names in alphabetical order:
Aashruti
--- Menu ---
1. Add name and age
2. Find people above a certain age
3. Display names alphabetically
4. Exit
Enter your choice: 4
Goodbye!
Process returned 0 (0x0) execution time : 26.997 s
Press any key to continue.

2. Stack Problem: Implement a stack using arrays (not STL) that:

    1.Has basic push and pop operations

    2.Has a function to find middle element

    3.Has a function to reverse only bottom half of stack

    4.Maintain stack size of 10

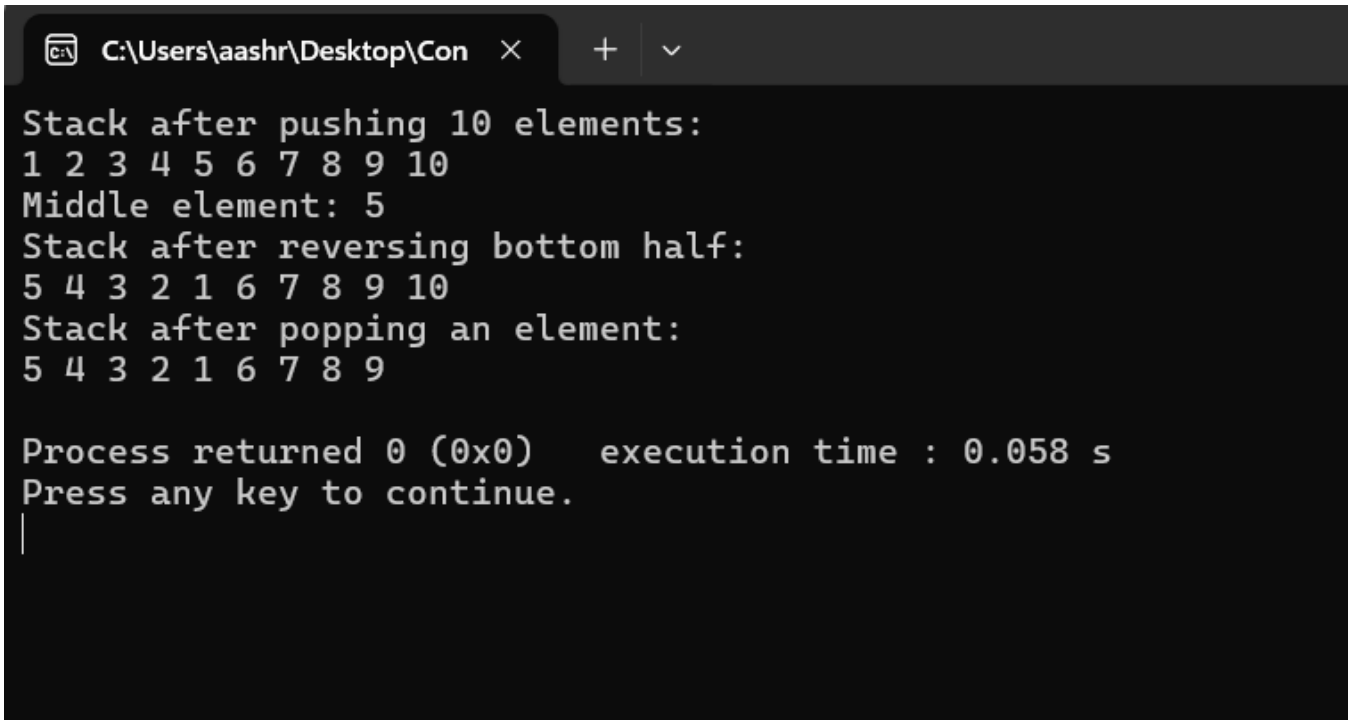
```
#include <iostream>

using namespace std;


class Stack {

private:

   int arr[10];

   int top;

public:

   Stack() {

      top = -1;

   }


   void push(int x) {

      if (top >= 9) {

         cout << "Stack overflow!" << endl;

      } else {

         arr[++top] = x;

      }

   }



   int pop() {

      if (top == -1) {

         cout << "Stack underflow!" << endl;
```

```
        return -1;
    } else {
        return arr[top--];
    }
}

int findMiddle() {
    if (top == -1) {
        cout << "Stack is empty!" << endl;
        return -1;
    }
    return arr[top / 2];
}



void reverseBottomHalf() {
    if (top == -1) {
        cout << "Stack is empty!" << endl;
        return;
    }
    int mid = top / 2;

    int start = 0;
    int end = mid;
    while (start < end) {
        swap(arr[start], arr[end]);
        start++;
        end--;
    }
```

```
    }



    void display() {
        if (top == -1) {
            cout << "Stack is empty!" << endl;
        } else {
            for (int i = 0; i <= top; i++) {
                cout << arr[i] << " ";
            }
            cout << endl;
        }
    }
};

int main() {
    Stack s;



    s.push(1);
    s.push(2);
    s.push(3);
    s.push(4);
    s.push(5);
    s.push(6);
    s.push(7);
    s.push(8);
    s.push(9);
    s.push(10);
```

```
    cout << "Stack after pushing 10 elements:" << endl;
    s.display();

    cout << "Middle element: " << s.findMiddle() << endl;

    s.reverseBottomHalf();
    cout << "Stack after reversing bottom half:" << endl;
    s.display();


    s.pop();
    cout << "Stack after popping an element:" << endl;
    s.display();

    return 0;
}
```

Output:


C:\Users\aashr\Desktop\Con
Stack after pushing 10 elements:
1 2 3 4 5 6 7 8 9 10
Middle element: 5
Stack after reversing bottom half:
5 4 3 2 1 6 7 8 9 10
Stack after popping an element:
5 4 3 2 1 6 7 8 9
Process returned 0 (0x0) execution time : 0.058 s
Press any key to continue.

3. Queue Problem: Implement a queue using arrays (not STL) that:

   1.Has basic enqueue and dequeue operations

   2.Has a function to reverse first K elements

   3.Has a function to interleave first half with second half

   4.Handle queue overflow/underflow

```
#include <iostream>
using namespace std;

class Queue {
private:
   int arr[10];
   int front;
   int rear;

public:
   Queue() {
      front = -1;
      rear = -1;
   }

   void enqueue(int x) {
      if ((rear + 1) % 10 == front) {
         cout << "Queue overflow!" << endl;
      } else {
         if (front == -1) {
            front = 0;
         }
         rear = (rear + 1) % 10;
```

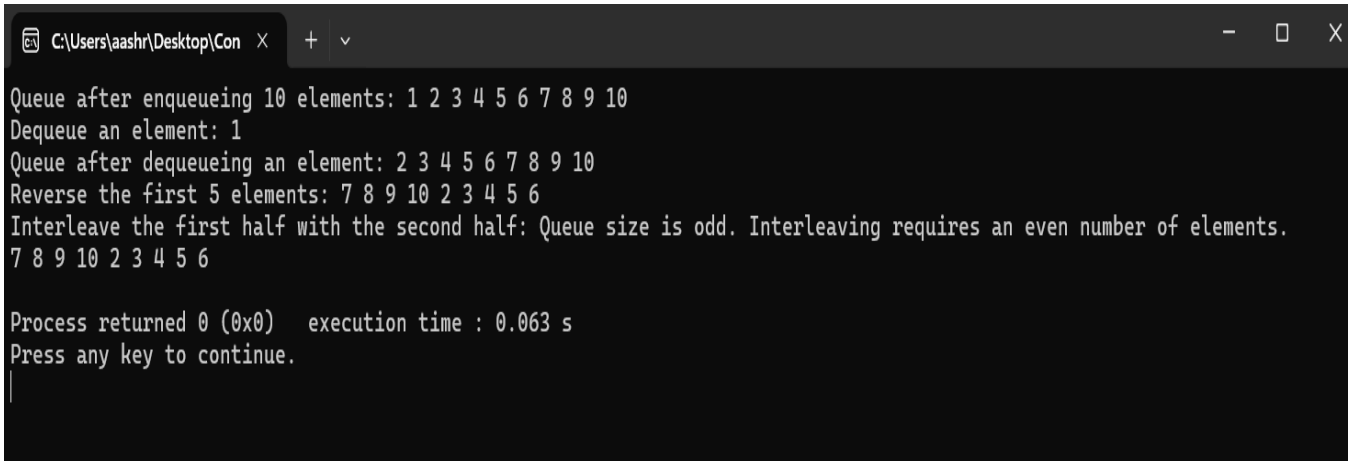
```
        arr[rear] = x;
    }
}

int dequeue() {
    if (front == -1) {
        cout << "Queue underflow!" << endl;
        return -1;
    } else {
        int val = arr[front];
        if (front == rear) {
            front = rear = -1;
        } else {
            front = (front + 1) % 10;
        }
        return val;
    }
}

void reverseFirstKElements(int k) {
    if (front == -1) {
        cout << "Queue is empty!" << endl;
        return;
    }
    if (k <= 0 || k > (rear - front + 1)) {
        cout << "Invalid value of K!" << endl;
        return;
    }
```

```
    int stack[k];
    int count = 0;
    while (count < k && front != -1) {
        stack[count++] = dequeue();
    }

    for (int i = 0; i < count; i++) {
        enqueue(stack[i]);
    }
}

void interleaveFirstHalfWithSecondHalf() {
    if (front == -1) {
        cout << "Queue is empty!" << endl;
        return;
    }

    int size = (rear - front + 1);
    if (size % 2 != 0) {
        cout << "Queue size is odd. Interleaving requires an even number of elements." << endl;
        return;
    }

    int mid = size / 2;
    int firstHalf[mid];
    int secondHalf[mid];

    for (int i = 0; i < mid; i++) {
        firstHalf[i] = dequeue();
```

```
        }

        for (int i = 0; i < mid; i++) {
            secondHalf[i] = dequeue();
        }

        for (int i = 0; i < mid; i++) {
            enqueue(firstHalf[i]);
            enqueue(secondHalf[i]);
        }
    }

    void display() {
        if (front == -1) {
            cout << "Queue is empty!" << endl;
        } else {
            int i = front;
            while (i != rear) {
                cout << arr[i] << " ";
                i = (i + 1) % 10;
            }
            cout << arr[rear] << endl;
        }
    }
};

int main() {
    Queue q;
```

```
    q.enqueue(1);
    q.enqueue(2);
    q.enqueue(3);
    q.enqueue(4);
    q.enqueue(5);
    q.enqueue(6);
    q.enqueue(7);
    q.enqueue(8);
    q.enqueue(9);
    q.enqueue(10);

    cout << "Queue after enqueueing 10 elements: ";
    q.display();

    cout << "Dequeue an element: " << q.dequeue() << endl;
    cout << "Queue after dequeueing an element: ";
    q.display();

    cout << "Reverse the first 5 elements: ";
    q.reverseFirstKElements(5);
    q.display();

    cout << "Interleave the first half with the second half: ";
    q.interleaveFirstHalfWithSecondHalf();
    q.display();

    return 0;
}
```

Output:


C:\Users\aashr\Desktop\Con
Queue after enqueueing 10 elements: 1 2 3 4 5 6 7 8 9 10
Dequeue an element: 1
Queue after dequeueing an element: 2 3 4 5 6 7 8 9 10
Reverse the first 5 elements: 7 8 9 10 2 3 4 5 6
Interleave the first half with the second half: Queue size is odd. Interleaving requires an even number of elements.
7 8 9 10 2 3 4 5 6
Process returned 0 (0x0) execution time : 0.063 s
Press any key to continue.

4. Linked List Problem: Create a singly linked list (not STL) that:

   1.Has functions to insert at start/end/position

   2.Has a function to detect and remove loops

   3.Has a function to find nth node from end

   4.Has a function to reverse list in groups of K nodes

```
#include <iostream>
using namespace std;

class Node {
public:
   int data;
   Node* next;
   Node(int val) {
      data = val;
      next = nullptr;
   }
};

class LinkedList {
private:
   Node* head;

public:
   LinkedList() {
      head = nullptr;
   }

   void insertAtStart(int val) {
```

```
    Node* newNode = new Node(val);
    newNode->next = head;
    head = newNode;
}

void insertAtEnd(int val) {
    Node* newNode = new Node(val);
    if (head == nullptr) {
        head = newNode;
    } else {
        Node* temp = head;
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}

void insertAtPosition(int val, int pos) {
    Node* newNode = new Node(val);
    if (pos == 1) {
        newNode->next = head;
        head = newNode;
        return;
    }
    Node* temp = head;
    for (int i = 1; temp != nullptr && i < pos - 1; i++) {
        temp = temp->next;
    }
```

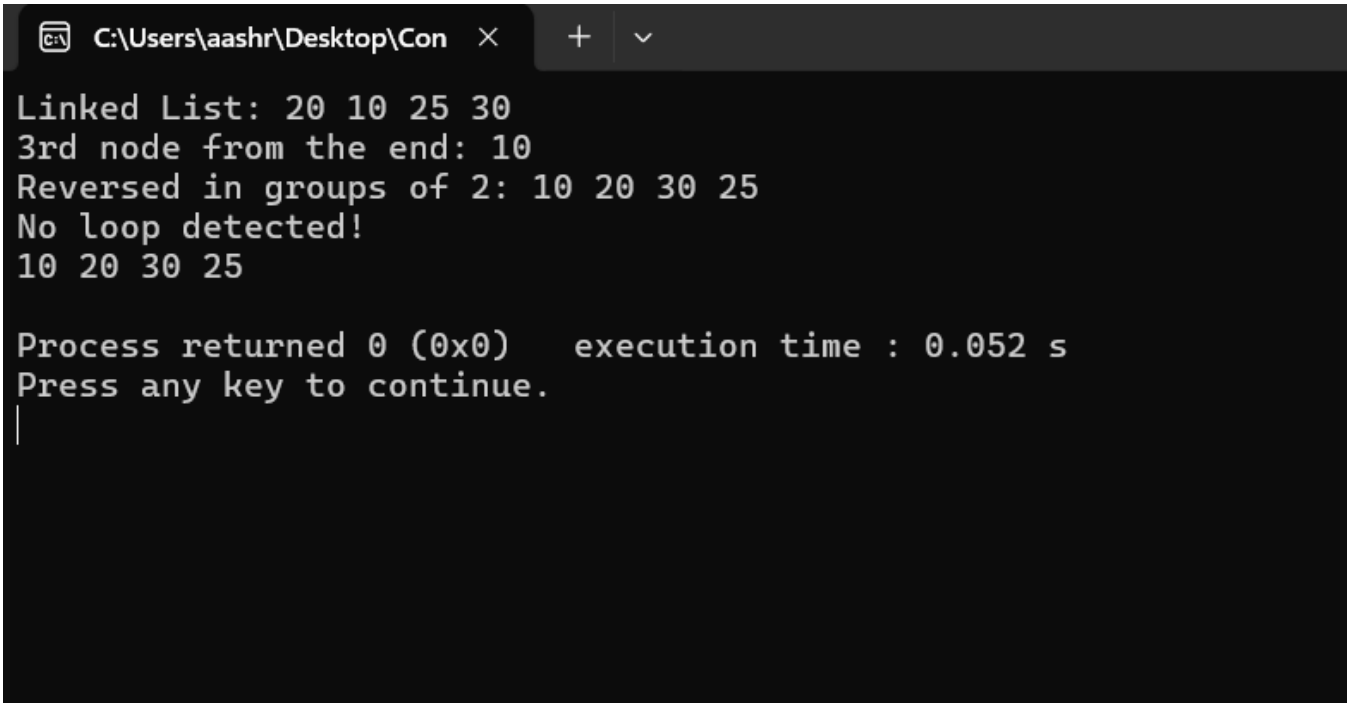
```
    if (temp == nullptr) {
        cout << "Position out of bounds!" << endl;
        return;
    }
    newNode->next = temp->next;
    temp->next = newNode;
}

void removeLoop() {
    Node* slow = head;
    Node* fast = head;
    while (fast != nullptr && fast->next != nullptr) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {
            cout << "Loop detected. Removing loop..." << endl;
            slow = head;
            while (slow->next != fast->next) {
                slow = slow->next;
                fast = fast->next;
            }
            fast->next = nullptr;
            return;
        }
    }
    cout << "No loop detected!" << endl;
}

int findNthFromEnd(int n) {
```

```
    Node* first = head;
    Node* second = head;
    for (int i = 0; i < n; i++) {
        if (second == nullptr) {
            cout << "N is greater than the number of nodes!" << endl;
            return -1;
        }
        second = second->next;
    }
    while (second != nullptr) {
        first = first->next;
        second = second->next;
    }
    return first->data;
}

void reverseInGroups(int k) {
    head = reverseInGroupsRecursive(head, k);
}

Node* reverseInGroupsRecursive(Node* head, int k) {
    Node* current = head;
    Node* prev = nullptr;
    Node* next = nullptr;
    int count = 0;

    while (current != nullptr && count < k) {
        next = current->next;
        current->next = prev;
```

```
            prev = current;
            current = next;
            count++;
        }

        if (next != nullptr) {
            head->next = reverseInGroupsRecursive(next, k);
        }

        return prev;
    }

    void display() {
        if (head == nullptr) {
            cout << "List is empty!" << endl;
            return;
        }
        Node* temp = head;
        while (temp != nullptr) {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
    }
};

int main() {
    LinkedList list;
    list.insertAtStart(10);
```

```
    list.insertAtStart(20);
    list.insertAtEnd(30);
    list.insertAtPosition(25, 3);
    cout << "Linked List: ";
    list.display();

    cout << "3rd node from the end: " << list.findNthFromEnd(3) << endl;

    list.reverseInGroups(2);
    cout << "Reversed in groups of 2: ";
    list.display();

    list.removeLoop();
    list.display();

    return 0;
}
```

Output:


C:\Users\aashr\Desktop\Con
Linked List: 20 10 25 30
3rd node from the end: 10
Reversed in groups of 2: 10 20 30 25
No loop detected!
10 20 30 25
Process returned 0 (0x0) execution time : 0.052 s
Press any key to continue.